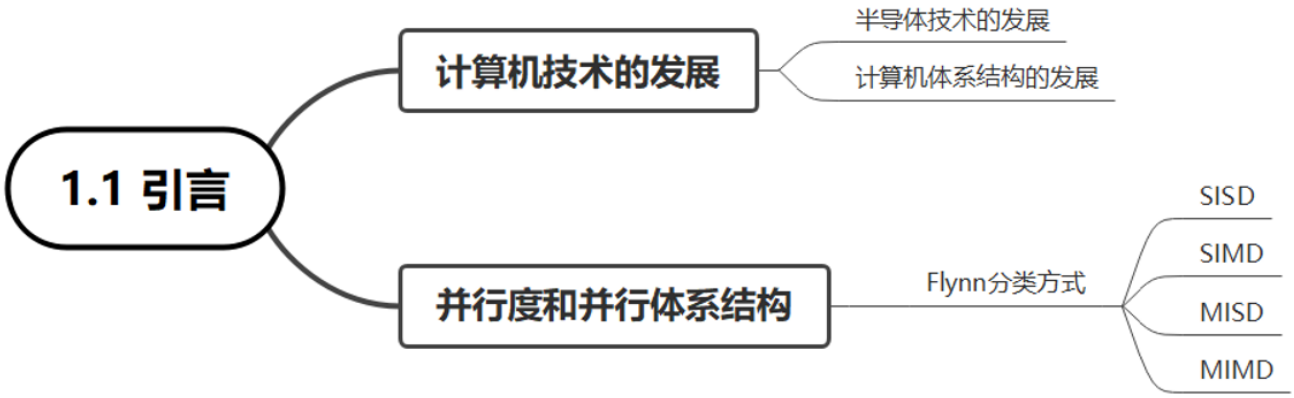
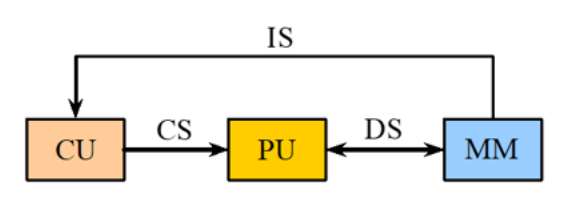
第1章 概述



1.1 并行度和并行体系结构（Flynn分类法）

- CU（Control Unit，**控制单元**）：负责从内存取指令、译码，生成控制信号。
- PU（Processing Unit，**处理单元**）：也就是运算器，负责执行算术 / 逻辑运算。
- MM（Main Memory，**主存储器**）：存放指令和数据。
- IS（Instruction Stream，**指令流**）：指令从内存流向控制单元的路径。
- CS（Control Signals，**控制信号**）：控制单元发给处理单元的执行命令。
- DS（Data Stream，**数据流**）：数据在主存和处理单元之间传输的路径。

单指令流单数据流，SISD

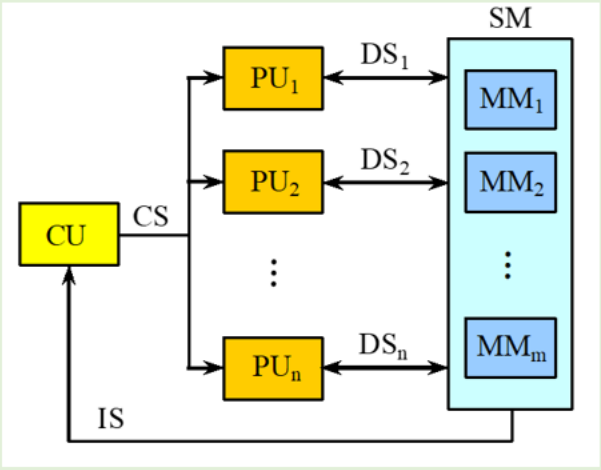


- 这个类别是单处理器；
- 可以使用指令级并行，如流水线、动态调度、超标量。

虽然它是“单指令流、单数据流”，但不代表它完全没有并行能力，它可以在**指令层面**实现并行：

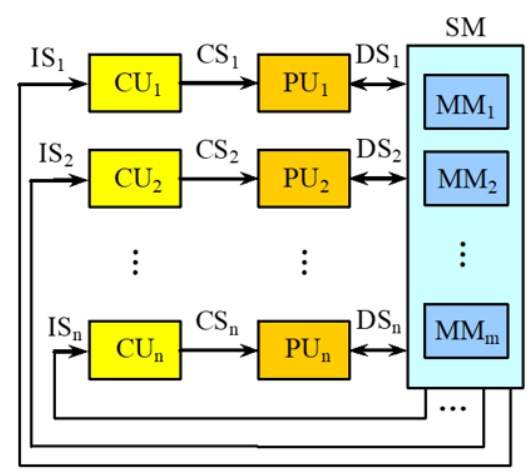
- **流水线（Pipeline）**：把一条指令拆成多个阶段，让不同指令的不同阶段重叠执行，比如取指、译码、执行、访存、写回五步，让第 1 条指令在执行时，第 2 条在译码，第 3 条在取指，相当于同时处理多条指令的不同部分。
- **动态调度（Dynamic Scheduling）**：硬件自动调整指令执行顺序，避免等待数据依赖造成的停顿，让流水线更高效。
- **超标量（Superscalar）**：让控制单元在一个时钟周期内，向多个独立的执行单元发射多条指令，相当于在单处理器内部，同时执行多条不相关的指令。

单指令流多数据流，SIMD



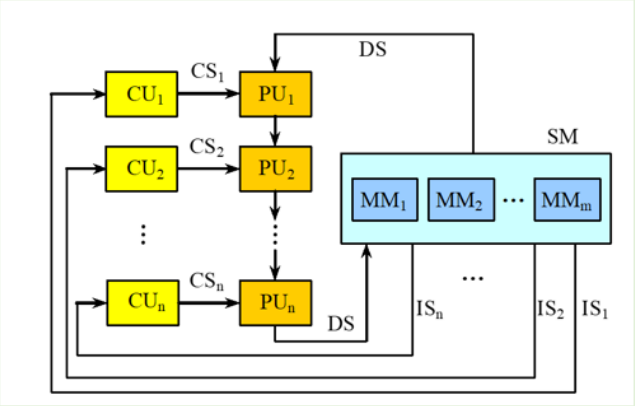
- 同一指令操纵多条数据流，由不同处理器，对多个数据项进行并行数据处理；
- 每个数据处理器可有自己的数据存储器，但是只有一个指令存储器和控制器；
- 向量体系结构，GPU

多指令流多数据流，MIMD



- 每个处理器都自己取指令，并对自己的数据进行处理；
- 任务级并行；
- 紧密耦合的MIMD结构—线程级并行；
- 松散耦合MIMD结构—集群。

多指令流单数据流， MISD



- 目前为止，没有此种类型的商用机；
- 脉动阵列。

1.2 计算机体系结构

1.2.1 两大核心定义

1、狭义定义（Amdahl 经典定义）

机器语言程序员所能看到的计算机全部属性，核心是**指令集（ISA）设计**。需考量核心要素：寄存器配置、内存地址规则、访存方式、指令集格式与功能。

2、广义定义（《计算机体系结构–量化研究方法》）

涵盖计算机设计全维度，包含三大核心层级：**指令集结构、计算机组成、硬件实现**。

核心本质：设计能够满足特定性能、功耗、成本目标的计算机组织架构与硬件实现方案。

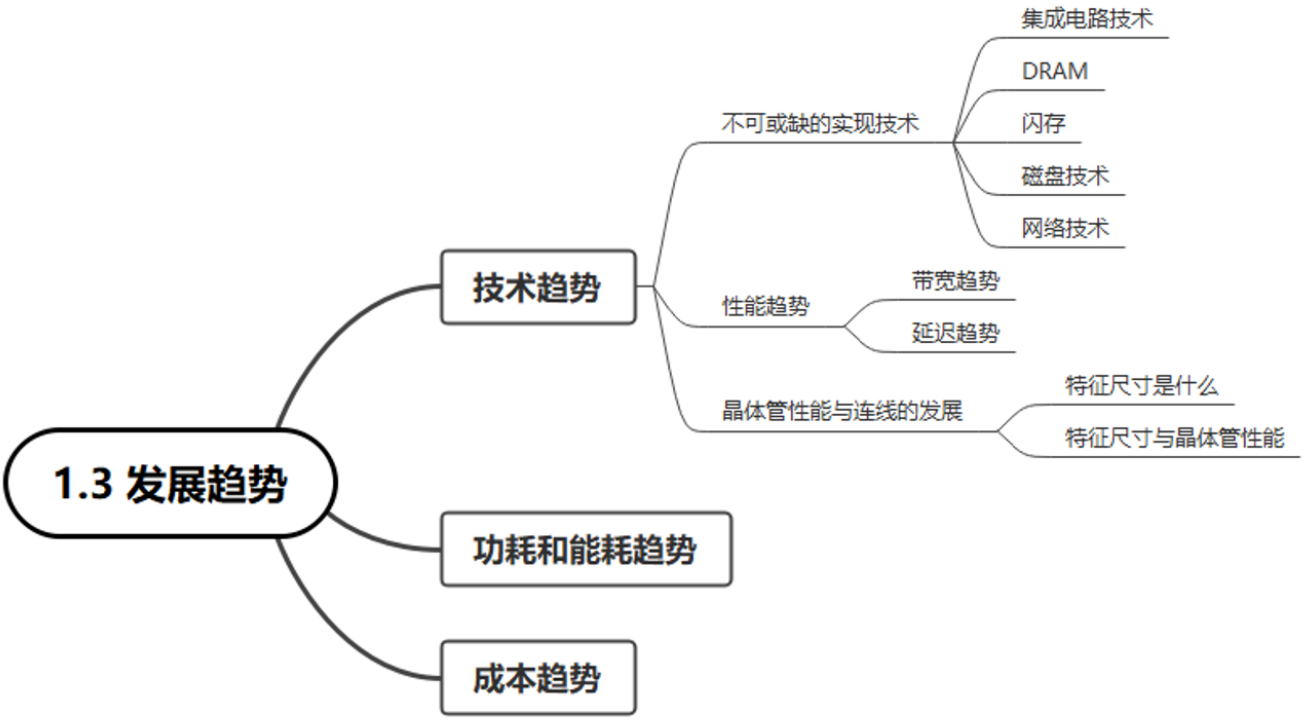
1.2.2 关键技术参数

- 存储器寻址：默认**字节寻址**，支持访问对齐机制
- 寻址模式：立即寻址、寄存器寻址、偏移寻址等
- 操作数：定义不同数据类型、数据存储大小
- 指令类型：数据传输、算术逻辑运算、控制跳转、浮点运算等
- 指令格式：固定长度（RISC架构）、可变长度（CISC架构）

1.2.3 组成与硬件的区别

- **计算机组成**：逻辑实现层面，包含存储系统架构、存储器互联方式、CPU内部逻辑设计，是ISA的具体逻辑落地。
- **计算机硬件**：物理实现层面，包含详尽的逻辑电路设计、芯片封装、元器件工艺等物理硬件细节。

1.3 发展趋势



1.3.1 性能趋势结论

带宽的改进要明显优于延迟。一个简单的经验法则是：带宽的增长速度至少是延迟改进速度的平方。

1.3.2 晶体管（Transistors）性能与连线（wire）的发展

特征尺寸就是一个晶体管或者一条连线在x方向或者y方向的最小尺寸。

随着特征尺寸缩小，连线会变短，但是单位长度的电阻和电容都会变差。和晶体管性能相比，连线延迟方面的改进少的可怜。

1.3.3 功率和能耗趋势

- 处理器需要的最大功率是多少？

如果处理器需要的最大功率超过电源系统能提供的功率，就会导致电压下降，而电压下降会导致期间无法正常工作；

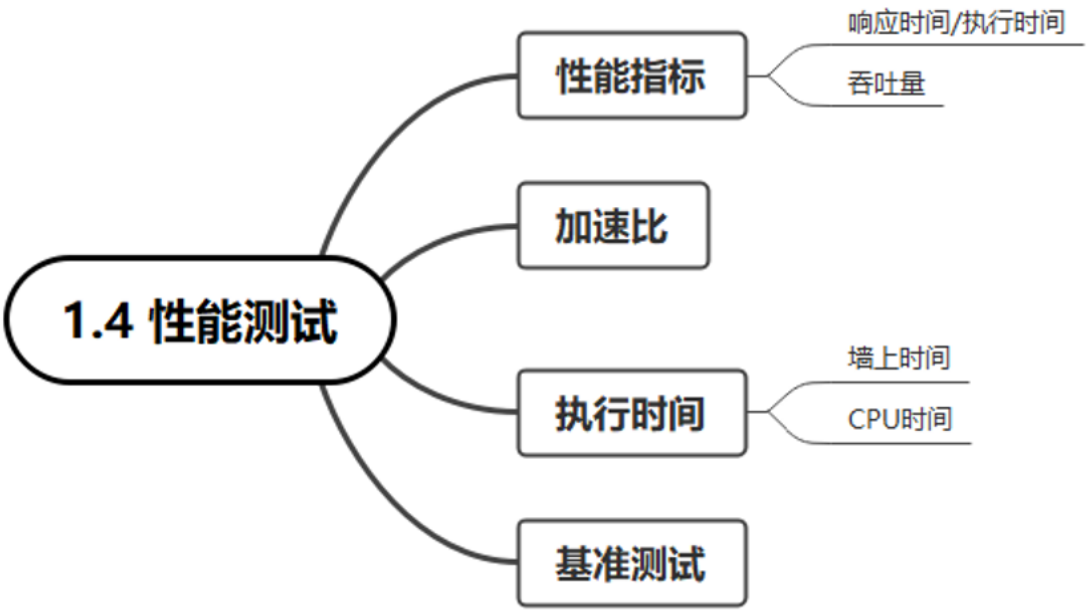
- 持续功耗 -- 热设计功耗 （Thermal Design Power，TDP）

决定了冷却需求，当温度接近节点温度上限时，电路降低时钟频率，从而减小功率；如果不成功，启动热过载启动装置，以降低芯片的功率。

- 能耗和能耗效率。

所谓的功率，就是单位时间的能耗。 用能耗更适合作为对比处理器的参数。因为能耗这个参数同时包含了执行任务的时间。 也就是，如果对比两个处理器对于某一给定任务，那个更高效，那么应该对比的是执行该项任务的能耗，而非功率。

1.4 性能测试



1.5 充分利用并行

同时性：两个或两个以上的事件在同一时刻发生。

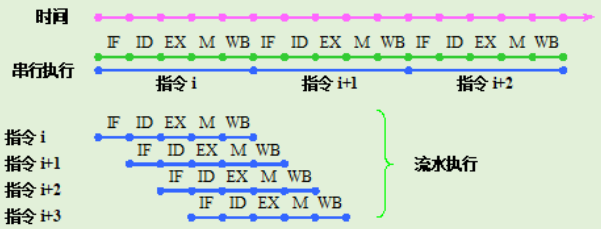
并发性：两个或两个以上的事件在同一时间间内发生。 （MIPS 流水线）

提高并行性的三种途径：

时间重叠： 引入时间因素，让多个处理过程在时间上相互错开，轮流重叠地使用同一套硬件设备的各个部分，以加快硬件周转而赢得速度。

时间重叠

- 单处理机中 -- 部件功能专用化
  - 把一件工作按功能分割为若干相互联系的部分；
  - 把每一部分指定给专门的部件完成；
  - 按时间重叠原理把各部分的执行过程在时间上重叠起来，使所有部件依次分工完成一组同样的工作。



- 多处理机中 – 处理机专用化
  - 专用外围处理机

    例如，输入/输出功能的分离。
  - 专用处理机

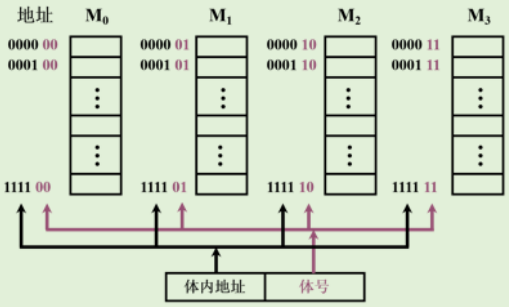
    如数组运算、高级语言翻译、数据库管理等，分离出来。
  - 异构型多处理机系统

    由多个不同类型、至少担负不同功能的处理机组成，它们按照作业要求的顺序，利用时间重叠原理，依次对它们的多个任务进行加工，各自完成规定的功能动作。

资源重复：引入空间因素，以数量取胜。通过重复设置硬件资源，大幅度地提高计算机系统的性能。

资源重复

- 单处理机中 -- 重复设置功能部件
  - 多存储体并行
  - 数据/指令独立存储体
  - 重复设置运算部件



低位交叉多体并行

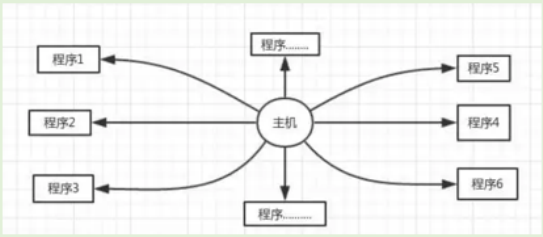
- 多处理机中 – 重复设置处理机
  - 容错系统
  - 同构型多处理机系统

    由多个同类型或至少担负同等功能的处理机组成，它们同时处理同一作业中能并行执行的多个任务。

资源共享：这是一种软件方法，它使多个任务按一定时间顺序轮流使用同一套硬件设备。如，多道程序。

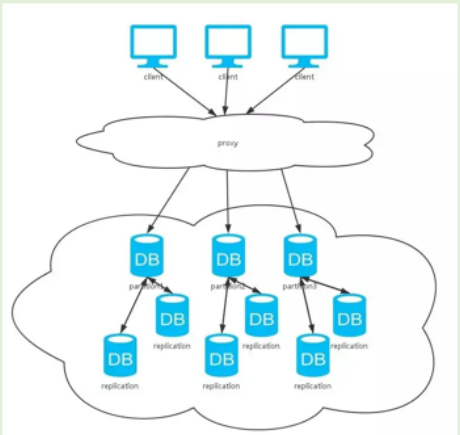
资源共享

- 单处理机中
  - 分时系统



提取文字 ••• 更多功能

- 多处理机中
  - 分布式系统



1.6 局部性原理

程序执行时所访问的存储器地址分布不是随机的，而是相对地簇聚。

常用的一个经验规则：程序执行时间的90%都是在执行程序中10%的代码。

程序的时间局部性：程序即将用到的信息很可能就是目前正在使用的信息。

程序的空间局部性：程序即将用到的信息很可能与目前正在使用的信息在空间上相邻或者临近。

1.7 重点关注常见情形

- 对经常发生的情况采用优化方法的原则进行选择，以得到更多的总体上的改进。
- 优化是指分配更多的资源、达到更高的性能或者分配更多的电能等。
- 例如：
  - 相对溢出，不溢出是经常性事件。针对不溢出的情况进行性能设计，无需花过多的时间和精力在解决溢出上。
  - 处理器的取值、译码比乘法器用的更加频繁，优先优化取指令和译码部件。

1.8 Amdahl 定律

$$\text{加速比} = \frac{\text{总执行时间}_{\text{改进前}}}{\text{总执行时间}_{\text{改进后}}}$$

$$= \frac{1}{(1-\text{可改进比例}) + \frac{\text{可改进比例}}{\text{部件加速比}}}$$

Amdahl定律表明：加快某部件执行速度所能获得的系统性能加速比，受限于该部件的执行时间占系统中总执行时间的百分比。

- Amdahl定律：一种性能改进的递减规则
  - 如果仅仅对计算任务中的一部分做性能改进，则改进得越多，所得到的总体性能的提升就越有限。
  - 重要推论：如果只针对整个任务的一部分进行改进和优化，那么所获得的加速比不超过

    1/（1－可改进比例）

公式扩展

假设：

- S_i为第 i 种部件的加速比；
- P_i为第 i 种部件执行时间占总执行时间中的比例。

$$\text{加速比} = \frac{1}{(1-\text{可改进比例}) + \frac{\text{可改进比例}}{\text{部件加速比}}}$$

$$= \frac{1}{(1-\sum P_i) + \sum \frac{P_i}{S_i}}$$

例题

例 将计算机系统中某一功能的处理速度提高到原来的20倍，但该功能的处理时间仅占整个系统运行时间的40%，则采用此提高性能的方法后，能使整个系统的性能提高多少？

解 由题可知，可改进比例 = 40% = 0.4，部件加速比 = 20

根据Amdahl定律可知：

$$\text{总加速比} = \frac{1}{0.6 + (0.4/20)} = 1.613$$

采用此提高性能的方法后，能使整个系统的性能提高到原来的1.613倍。

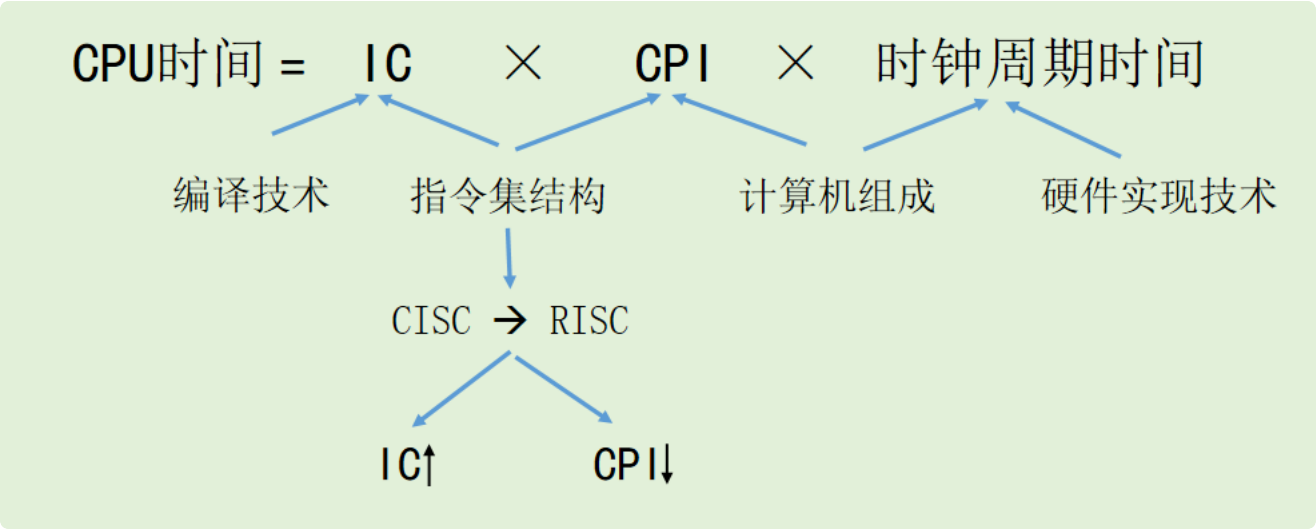
1.9 CPU性能公式

CPU时间

= 执行程序所需的时钟周期数×时钟周期时间

= IC ×CPI ×时钟周期时间CPU时间

= IC ×CPI ×（1/时钟频率）



公式扩展

- CPI时间公式拓展

假设：计算机系统有n种指令，

CPI_i ：第i种指令的处理时间（周期数）；

IC_i ：在程序中第i种指令出现的次数；

则

$$\text{CPU时间} = \text{执行程序所需的时钟周期数} \times \text{时钟周期时间}$$
$$= \sum_{i=1}^{n} (CPI_i \times IC_i) \times \text{时钟周期时间}$$

- CPI公式拓展

$$CPI = \frac{\text{时钟周期数}}{IC} = \frac{\sum_{i=1}^{n} (CPI_i \times IC_i)}{IC} = \sum_{i=1}^{n} (CPI_i \times \frac{IC_i}{IC})$$

其中，(IC_i/IC)反映了第i种指令在程序中所占的比例。

例题

例　考虑条件分支指令的两种不同设计方法：

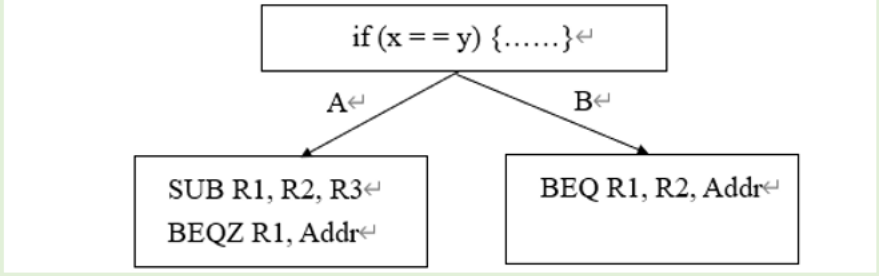
（1）CPU_A：通过比较指令设置条件码，然后测试条件码进行分支。

（2）CPU_B：在分支指令中包括比较过程。

在这两种CPU中，条件分支指令都占用2个时钟周期，而所有其他指令占用1个时钟周期。对于CPU_A，执行的指令中分支指令占20%；由于每条分支指令之前都需要有比较指令，因此比较指令也占20%。由于CPU_A在分支时不需要比较，因此CPU_B的时钟周期时间是CPU_A的1.25倍。问：哪一个CPU更快？

若CPU_B的时钟周期时间是CPU_A的1.1倍呢？（自己完成）

例如



分 析

CPU_A：

- 需要两条指令完成分支，比较置条件码指令 + 分支指令。
- 假设完成某一功能F，使用CPUA，程序需100条指令；

程序指令构成：

指令类型	CPI_i	指令数
比较置位	1	20
测试分支	2	20
其　　他	1	60

CPU_B：

- 需要一条指令完成比较和分支
- 假设完成同样功能F，使用CPU_B，程序需80条指令；

程序指令构成：

指令类型	CPI_i	指令数
比较并分支	2	20
其　　他	1	60

解

我们不考虑所有系统问题，所以可用CPU性能公式。占用2个时钟周期的分支指令占总指令的20%，剩下的指令占用1个时钟周期。所以

$$CPI_A = 0.2 \times 2 + 0.80 \times 1 = 1.2$$

则CPU_A性能为

$$总CPU时间_A = IC_A \times 1.2 \times 时钟周期_A$$

在CPU_B中没有独立的比较指令，所以CPU_B的程序量为CPU_A的80%，分支指令的比例为

$$20\%/80\% = 25\%$$

这些分支指令占用2个时钟周期，而剩下的75%的指令占用1个时钟周期，因此

$$CPI_B = 0.25 \times 2 + 0.75 \times 1 = 1.25$$

因为CPU_B不执行比较，故

$$IC_B = 0.8 \times IC_A$$

同时，根据假设，有

$$时钟周期_B = 1.25 \times 时钟周期_A$$

因此CPU_B性能为

$$总CPU时间_B = IC_B \times CPI_B \times 时钟周期_B$$

$$= 0.8 \times IC_A \times 1.25 \times (1.25 \times 时钟周期_A)$$

$$= 1.25 \times IC_A \times 时钟周期_A$$

在这些假设之下，尽管CPU_B执行指令条数较少，CPU_A因为有着更短的时钟周期，所以比CPU_B快。

1.10 摩尔定律、Dennard定律

1.5.6 摩尔定律

摩尔定律是由英特尔(Intel)名誉董事长戈登·摩尔(Gordon moore)经过长期观察总结的经验 。被称为计算机第一定律。摩尔定律是指单芯片上可容纳的晶体管数目，约每隔18个月便会增加一倍，性能也将提升一倍。

1.5.7 Dennard定律

晶体管面积的缩小使得其所消耗的电压以及电流会以差不多相同的比例缩小。也就是说，如果晶体管的大小减半，该晶体管的静态功耗将会降至四分之一（电压电流同时减半）。